Scalable ETL and MLOps Pipeline

Python Data Processing and Model Deployment

Student name: _____

Course: _____

Instructor: _____

Submission date: _____

Pipeline Design and Data Sources

A reliable machine learning service begins with repeatable data preparation. This project implements a customer churn workflow that extracts records, creates consistent features, stores a validated snapshot, trains a classifier, and packages predictions behind a REST API. Python connects the stages, while Airflow scheduling and GitHub Actions provide complementary forms of automation. The design favors a runnable single-host lab while making its scaling boundaries explicit.

The demonstration uses 2,400 synthetic customers generated with a fixed random seed. A CSV file contains 1,200 records, a SQLite source table contains 800, and an offline JSON API snapshot contains 400. Each source supplies an identifier, tenure, monthly charge, free-text review, and binary churn label. The text field represents unstructured content inside an otherwise structured record. Synthetic data allow repeatable testing without exposing personal customer information.

Extraction reads CSV and SQL data in batches of 10,000 rows. SQLAlchemy manages database connections, and requests supports a live HTTP source when SOURCE_API_URL is configured. That endpoint must return records and a next-page token. A timeout, unsuccessful status, or repeated token stops extraction instead of silently producing an incomplete snapshot. Offline execution uses the saved JSON source, so it does not depend on a public API remaining available.

Extracted batches become temporary CSV partitions. Dask then schedules partition-level cleaning, and SQLAlchemy publishes the result to a separate SQLite warehouse. Training reads only the published table, and the API loads a saved preprocessing and model pipeline. This dependency chain keeps invalid intermediate data away from downstream consumers. The design deliberately separates source records, processed data, experiment history, and deployable artifacts so each stage has a clear input and output.

Transformation Loading and Scalability

Transformation converts numeric fields with pandas and removes rows whose target is missing or outside the binary label contract. Negative tenure and charges become missing values rather than plausible measurements. Empty reviews become empty strings, and their word counts provide a simple numerical text feature. This feature is inexpensive and inspectable, although it cannot capture sentiment or the meaning of a complaint.

Missing numerical values remain in the warehouse until training. Median imputation and standardization belong inside the fitted scikit-learn pipeline, where they learn parameters only from training data. Computing these statistics over the full dataset before splitting would expose test information. The scikit-learn documentation recommends consistent preprocessing and pipelines to avoid these errors (scikit-learn developers, n.d.). The same fitted transformations therefore accompany the model into serving.

Dask reads staged CSV data with a 16 MiB block size and computes groups of at most four partitions with its threaded scheduler. Each partition uses pandas operations. The database sink writes batches sequentially, keeping only a bounded group of results in memory. Dask guidance emphasizes balancing partition size against memory and scheduler overhead rather than assuming that more partitions always improve speed (Dask developers, n.d.). No throughput or cluster-scale benchmark is claimed for this small example.

The warehouse loader creates a staging table, enforces a primary key, checks the valid row count, and replaces the published table in a transaction. Duplicate identifiers cause failure; they are not silently discarded. Repeating a successful load replaces the snapshot without duplicating records. SQLite still limits concurrent writers, and the staging files require a single active producer. Scaling beyond this lab would require immutable interval-specific storage, a shared distributed scheduler, and a database designed for concurrent loading. These changes address storage and coordination limits that parallel transformations alone cannot solve.

Model Training Tracking and Serving

The training stage uses logistic regression to estimate churn probability from tenure, monthly charge, and review word count. It separates an 80 percent training portion from a 20 percent holdout using stratification and a fixed random seed. Within the training portion, three-fold cross-validation compares regularization strengths of 0.1, 1, and 10. Each fold fits its own imputer and scaler, preserving the separation between model selection and holdout evaluation.

MLflow records candidate cross-validation AUC values in nested runs. The parent run stores the selected parameters, holdout AUC and accuracy, dataset fingerprint, model signature, and fitted model. CI also attaches the Git commit identifier. MLflow Tracking organizes parameters, metrics, and artifacts around runs, providing the traceability needed to compare experiments (MLflow developers, n.d.). A SQLite tracking backend supports the local exercise; a shared service would be needed for a team.

A holdout AUC below 0.65 prevents publication of the local serving artifact. This threshold is an instructional gate rather than evidence of a production-ready classifier. A real promotion decision would also consider calibration, subgroup performance, business costs, and an independently maintained evaluation set. The training query caps data at 100,000 ordered records to bound memory. That selection could bias a larger dataset and should be replaced with representative sampling or an incremental learning method.

The fitted pipeline is serialized with joblib and paired with metadata containing its MLflow run identifier. Flask exposes prediction, health, and metrics endpoints; Waitress supplies the container HTTP server. Prediction requests must contain exactly three finite nonnegative numeric features. Responses include churn probability and run identifier, allowing consumers to identify the serving version. Because the model loads at startup, artifact changes require a service restart. Packaging both files in an immutable image keeps preprocessing and prediction behavior aligned during deployment.

Workflow Automation and Deployment

Airflow and GitHub Actions automate different triggers. The Airflow DAG schedules ETL followed by training each day, allows two retries, disables historical catchup, and permits one active run. It invokes a separate Python environment through Bash tasks so the core application dependencies do not need to share Airflow's environment. Linux or WSL2 provides the Airflow runtime, while the basic pipeline can run directly on Windows.

The DAG assumes a single host with shared project storage. Source snapshots must remain fixed during retries, because rereading changed input could produce a different model. Airflow documentation recommends repeatable task outcomes, partition-specific inputs, and durable storage for communication across workers (Apache Software Foundation, n.d.). The lab's single-run constraint reduces local collisions, but a distributed deployment would require run-specific paths and explicit data lineage instead of shared mutable files.

GitHub Actions installs dependencies, executes integration tests, and builds a fresh model. A main-branch push then builds a container, checks its health endpoint, and publishes an image tagged with the commit SHA. Pull requests run validation without publishing an image. This workflow reduces repeated manual packaging and makes the code revision associated with an image explicit. It does not prove deployment success until the resulting container has run in the target environment.

A separate deployment workflow accepts a tested immutable image tag and runs on a configured self-hosted Linux runner. It checks a candidate container before replacing the active service and restores the previous image if the replacement fails its health check. This lab strategy has brief downtime and requires a provisioned host and registry access. Scheduled Airflow training publishes eligible model files but does not independently restart the API. Linking scheduled model promotion to container release would be a further integration step, governed by evaluation and approval policies appropriate to the application.

Monitoring Validation and Lessons

Operational monitoring focuses on whether predictions are available and timely. The API exposes request counters by outcome, a latency histogram, and a histogram of predicted probabilities. Prometheus scrapes these measurements, and a provisioned Grafana dashboard shows traffic, failed or invalid requests, p95 latency, and probability distributions. Prometheus instrumentation guidance identifies requests, errors, and latency as central measurements for an online service (Prometheus authors, n.d.). Labels use bounded outcome values rather than customer identifiers.

Pipeline logs report loaded row counts and elapsed ETL time. Airflow records task failures and retries, while MLflow preserves training results. These records answer different questions: whether the workflow finished, whether an experiment met its quality gate, and whether the serving endpoint responds. Probability shifts may justify investigation, but they do not measure predictive accuracy. Evaluating model degradation requires eventual ground-truth labels and a defined comparison window; automated drift alerts and label-based monitoring remain extensions.

Validation targets behavior that could damage reliability. All three local tests passed. The offline run loaded 2,400 records and achieved holdout AUC of 0.777 and accuracy of 0.752 on 480 synthetic examples. Tests repeat loading to check idempotence, train the model, request predictions, and check input rejection and metrics exposure. A deliberate duplicate identifier fails loading while preserving the published table. Separate tests exercise API pagination and timeout handling with mocked responses. These results demonstrate local correctness, not performance on real customers.

The main design challenges are preventing preprocessing leakage, preserving valid data during failed loads, and defining an honest scaling boundary. Fitting transformations inside the model pipeline addresses leakage; transactional publication addresses partial loads; bounded partitions address ETL memory use. MLOps improves deployment efficiency by preserving experiment lineage, automating repeatable validation, and packaging a known model with its runtime. These benefits are architectural, not measured time savings. The accompanying validation record distinguishes local execution from Airflow, container, and hosted workflow checks that require their own environments.

References

Apache Software Foundation. (n.d.). Best practices. Apache Airflow documentation. https://airflow.apache.org/docs/apache-airflow/stable/best-practices.html

Dask developers. (n.d.). Dask DataFrames best practices. https://docs.dask.org/en/latest/dataframe-best-practices.html

MLflow developers. (n.d.). ML experiment tracking. https://www.mlflow.org/docs/latest/tracking/

Prometheus authors. (n.d.). Instrumentation. https://prometheus.io/docs/practices/instrumentation/

scikit-learn developers. (n.d.). Common pitfalls and recommended practices. https://scikit-learn.org/stable/common_pitfalls.html